



Context Course documentation

Quiz 1: Plugin Fundamentals ▾



Quiz 1: Plugin Fundamentals

[Copy page](#)

Test your understanding of plugins, their anatomy, and how they differ across platforms.

Question 1: What is a plugin?

- ☐ A plugin is a single reusable workflow, like calculating ROI
- ☐ A plugin is a collection of tools that provides standardized access to external APIs
- ☒ A plugin is a reusable extension surface; Claude Code and Codex use plugin bundles, while OpenCode plugins are code modules loaded from files or npm

Correct! Correct! The exact shape depends on the platform. Claude Code and Codex are manifest-first, while OpenCode plugins are code-first.

- ☐ A plugin is a text file that describes how to use an agent tool

[Submit](#)

You got all the answers!

Question 2: What does plugin.json do?

- ☒ For Claude Code and Codex, plugin.json tells the agent the plugin's identity and where to find bundled components

Correct! Correct! On manifest-first platforms, plugin.json identifies the plugin and points to skills, MCP config, or app integrations. OpenCode does not use this manifest model.

- ☐ plugin.json is just documentation describing what's in the plugin
- ☐ plugin.json is a list of all the API endpoints the plugin provides
- ☐ plugin.json is automatically generated and does not need to be maintained

[Ask HuggingChat](#)

You got all the answers!

Question 3: How do manifest-first plugins differ by platform?

- ☒ Claude Code uses `.claude-plugin/plugin.json`; Codex uses `.codex-plugin/plugin.json`; both place `skills/` at the plugin root

Correct! Correct! Both platforms use a similar pattern (manifest in a hidden directory, `skills` at root) but with platform-specific directory names.

- ☐ All platforms use the exact same `manifest.json` format at the plugin root
- ☐ Only Codex supports plugin marketplaces
- ☐ Claude Code and Codex plugins are fully interchangeable

You got all the answers!

Question 4: How do skills and MCP servers fit inside a plugin?

- ☐ In Claude Code and Codex, a plugin can only contain skills OR MCP servers, not both
- ☒ In Claude Code and Codex, skills are human-readable workflows; MCP servers provide programmatic tool access

Correct! Correct! Skills describe how to accomplish tasks; MCP servers provide standardized tool interfaces for agents to call.

- ☐ Skills and MCP servers do the same thing; you only need one
- ☐ Only Claude Code plugins use skills; Codex plugins only use MCP servers

You got all the answers!

Question 5: What can a Codex plugin reference?

- ☐ The plugin manifest directory is always named `.claude-plugin/` regardless of platform
- ☒ Codex plugins can reference MCP servers and agents



Ask HuggingChat



Correct! Correct! Codex plugin.json uses fields like 'mcpServers': './.mcp.json' and 'apps': './.app.json' to reference bundled configuration files.

- OpenCode uses a formal plugin manifest like Claude Code and Codex
- All plugins must have an MCP server to be useful

Submit

You got all the answers!

Summary

If you got 4-5 correct, you have a solid grasp of plugin anatomy. If you missed several, reread the introduction and anatomy lessons before moving on.

Key Takeaways

- Plugins are a packaging surface, not the same thing as a skill or an MCP server
- Claude Code and Codex use manifest-first plugins, while OpenCode uses code-first modules
- On manifest-first platforms, `plugin.json` tells the agent where bundled components live

Next Steps

Next, use plugins in a real workflow and see how installation and activation differ by platform.

[Update](#) on GitHub

← Building Your Own Plugin

Using Plugins →

Ask HuggingChat

